React.js

Form Handling + CRUD Notes
with Spring Boot + Axios Integration

- ✥ Controlled Inputs
- ✥ Submit Events
- ✥ POST Request with Axios
- ✥ Programmatic Redirects
- ✥ Deleting Stuff
- ✥ Not Found Route

Spring Boot + Axios backend integration

01 Controlled Inputs (Forms)

In React, form elements are usually made **controlled** by storing their values in state. That means the **input value always comes from state**, and every keystroke updates state through an **onChange** handler.

- Use **useState** for each field you want to control.
- Bind the field with **value={...}** and update it with **onChange**.
- For a **textarea** and **select**, the pattern is the same.
- Controlled inputs make validation, reset, and submit logic much easier.

JSX

```
import { useState } from "react";

const Create = () => {
  const [title, setTitle] = useState('');
  const [body, setBody] = useState('');
  const [author, setAuthor] = useState('mario');

  return (
    <div className="create">
      <h2>Add a New Blog</h2>
      <form>
        <label>Blog title:</label>
        <input
          type="text"
          required
          value={title}
          onChange={(e) => setTitle(e.target.value)}
        />
        <label>Blog body:</label>
        <textarea
          required
          value={body}
          onChange={(e) => setBody(e.target.value)}
        ></textarea>
        <label>Blog author:</label>
        <select
          value={author}
          onChange={(e) => setAuthor(e.target.value)}
        >
          <option value="mario">mario</option>
          <option value="yoshi">yoshi</option>
        </select>
        <button>Add Blog</button>
      </form>
    </div>
  );
}

export default Create;
```

Why controlled inputs matter

Because the state is the single source of truth, the UI and your data stay in sync. That makes it easy to clear fields after submit or validate before sending.

02 Submit Events

Forms submit naturally when the user clicks a button of type **submit** or presses Enter. In React, the form submit handler usually receives the event object so you can call **e.preventDefault()** and stop the browser from reloading the page.

- Attach the handler to the **form** with **onSubmit**, not just the button.
- **e.preventDefault()** keeps React in control of the page.
- Collect the input values from state and build an object.

JSX

```
import { useState } from "react";

const Create = () => {
  const [title, setTitle] = useState('');
  const [body, setBody] = useState('');
  const [author, setAuthor] = useState('mario');

  const handleSubmit = (e) => {
    e.preventDefault();
    const blog = { title, body, author };

    console.log(blog);
  }

  return (
    <div className="create">
      <h2>Add a New Blog</h2>
      <form onSubmit={handleSubmit}>
        <label>Blog title:</label>
        <input
          type="text"
          required
          value={title}
          onChange={(e) => setTitle(e.target.value)}
        />
        <label>Blog body:</label>
        <textarea
          required
          value={body}
          onChange={(e) => setBody(e.target.value)}
        ></textarea>
        <label>Blog author:</label>
        <select
          value={author}
          onChange={(e) => setAuthor(e.target.value)}
        >
          <option value="mario">mario</option>
          <option value="yoshi">yoshi</option>
        </select>
        <button>Add Blog</button>
      </form>
    </div>
  );
}

export default Create;
```

Button behavior

Inside a form, a plain **<button>** acts like submit by default. That is why the submit handler runs when the button is clicked.

03 Making a POST Request with Axios

For a Spring Boot backend, **Axios** is a clean choice for sending JSON. You build the blog object in state, send it to the backend, and let Axios handle the request body and response parsing.

- Use **axios.post(url, data)** for creation requests.
- Spring Boot typically expects JSON with **Content-Type: application/json**.
- Axios serializes the object for you.

JSX

```
import { useState } from "react";
import axios from "axios";

const Create = () => {
  const [title, setTitle] = useState('');
  const [body, setBody] = useState('');
  const [author, setAuthor] = useState('mario');

  const handleSubmit = (e) => {
    e.preventDefault();
    const blog = { title, body, author };

    axios.post("http://localhost:8000/blogs/", blog)
      .then(() => {
        console.log("new blog added");
      });
  }

  return (
    <div className="create">
      <h2>Add a New Blog</h2>
      <form onSubmit={handleSubmit}>
        ...
      </form>
    </div>
  );
}

export default Create;
```

Axios instead of fetch

Axios reads naturally with Spring Boot: **axios.post(..., blog)** is shorter than a fetch request with method, headers, and JSON.stringify.

It also keeps the same style you already used for GET requests in the previous notes.

04 Programmatic Redirects

Sometimes you do not want to navigate with a link. After a successful POST, for example, you may want to send the user back to the homepage automatically. That is a **programmatic redirect**.

- In React Router v5, use **useHistory()**.
- Call **history.push("/")** to go to a new route.
- **history.go(-1)** behaves like the browser back button.
- Do the redirect after the request succeeds.

JSX

```
import { useState } from "react";
import { useHistory } from "react-router-dom";
import axios from "axios";

const Create = () => {
  const [title, setTitle] = useState('');
  const [body, setBody] = useState('');
  const [author, setAuthor] = useState('mario');
  const history = useHistory();

  const handleSubmit = (e) => {
    e.preventDefault();
    const blog = { title, body, author };

    axios.post("http://localhost:8000/blogs/", blog)
      .then(() => {
        history.push("/");
      });
  }

  return (
    <div className="create">
      <h2>Add a New Blog</h2>
      <form onSubmit={handleSubmit}>
        ...
      </form>
    </div>
  );
}

export default Create;
```

Redirect flow

The request finishes first, then navigation happens. That prevents the page from leaving before the backend has saved the new blog.

05 Deleting Stuff

Deleting an item usually starts from a detail page or a list item. You fetch the item, show it, and attach a delete button. When the delete request succeeds, you redirect away so the user does not stay on a removed record.

- Use **axios.delete(url)** for delete requests.
- You can combine **useParams** and a custom fetch hook to load a single item.
- After deletion, redirect back to the homepage or previous page.

JSX

```
import { useHistory, useParams } from "react-router-dom";
import axios from "axios";
import useFetch from "./useFetch";

const BlogDetails = () => {
  const { id } = useParams();
  const { data: blog, error, isPending } = useFetch("http://localhost:8000/blogs/" + id);
  const history = useHistory();

  const handleClick = () => {
    axios.delete("http://localhost:8000/blogs/" + blog.id)
      .then(() => {
        history.push("/");
      });
  }

  return (
    <div className="blog-details">
      { isPending && <div>Loading...</div> }
      { error && <div>{ error }</div> }
      { blog && (
        <article>
          <h2>{ blog.title }</h2>
          <p>Written by { blog.author }</p>
          <div>{ blog.body }</div>
          <button onClick={handleClick}>delete</button>
        </article>
      )}
    </div>
  );
}

export default BlogDetails;
```

Important detail

If you keep the delete button on the detail page, the redirect happens only after the backend confirms the item was removed.

06 Not Found Route

A wildcard route catches any URL that does not match your defined routes. This is a nice way to show a friendly error page instead of leaving the user on a blank screen.

- Place the wildcard route at the end of your **Switch**.
- Render a small **NotFound** component.
- Use a **Link** to guide users back home.

JSX

```
import { Link } from "react-router-dom"

const NotFound = () => {
  return (
    <div className="not-found">
      <h2>Sorry</h2>
      <p>That page cannot be found</p>
      <Link to="/">Back to the homepage...</Link>
    </div>
  );
}

export default NotFound;

// App routes
<Route path="*">
  <NotFound />
</Route>
```

Good UX

A not-found page is especially useful after redirects and deletions, because it gives users a recovery path when they land on an invalid URL.